# cuGMEC Parameter Table

This document corresponds to `src/cuGMEC_param.h`. You must recompile after changing these parameters.

The Example column shows only one possible way to write each parameter; it does not imply a recommended value.

Notation:

| Notation | Meaning |
|---|---|
| `<Species>` | `Ion`, `Alpha`, `Beam` |
| `<MHDField>` | `Phi`, `A`, `dNe`, `dTe` |
| `<PICField>` | `dPi`, `dPa`, `dPb` |

# Data Types and File Paths

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `mhdReal` | `float` / `double` | MHD calculation precision. | MHD precision should be greater than or equal to PIC precision. | `using mhdReal = double;` |
| `picReal` | `float` / `double` | PIC calculation precision. | PIC precision should be less than or equal to MHD precision. | `using picReal = float;` |
| `inputDir` | String path | Input file directory. | For a run from scratch, only the `MHDEquilibrium` file is needed.<br>For a continued run, the `MHDContinue` and `PICContinue` files are also needed. They come from the previous run's `outputDir/final`. | `const std::string inputDir = "/path/to/input";` |
| `outputDir` | String path | Output file directory. | | `const std::string outputDir = "/path/to/output";` |
| `MHDEquilibrium` | `.bin` file name | MHD equilibrium file. | | `const std::string MHDEquilibrium = "MHDEquilibrium_384_32.bin";` |
| `<Species>PhaseSpaceMapping` | `.bin` file name | Phase-space orbit mapping file. | Required only when `ifOutputPhaseSpaceOrbit=trueType`. | `const std::string IonPhaseSpaceMapping = "IonPhaseSpaceMapping.bin";` |

# Normalization Parameters

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `B0` | Positive real number | Magnetic-field normalization scale. | | `const double B0 = 4.921751144619735;` |
| `L0` | Positive real number | Length normalization scale. | | `const double L0 = 6.595629295925759;` |
| `VA0` | Positive real number | Velocity normalization scale. | | `const double VA0 = 8.864164667700194e+06;` |
| `RHO0` | Positive real number | Left endpoint of the radial interval. | | `const double RHO0 = 0.08;` |
| `RHO1` | Positive real number | Right endpoint of the radial interval. | | `const double RHO1 = 0.90;` |
| `PSITMAX` | Positive real number | Outermost toroidal magnetic flux (Wb/rad). | | `const double PSITMAX = 18.868213504765762;` |

# Grid and Parallel Scale

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `hostNums` | Positive integer | Number of MPI processes. | | `const int hostNums = 4;` |
| `devNums` | Positive integer | Number of GPUs used by each MPI process. | | `const int devNums = 4;` |
| `gridNx` | Positive integer | Number of radial grid points. | | `const int gridNx = 256;` |
| `gridNy` | Positive integer | Number of field-aligned grid points. | Must be divisible by `hostNums * devNums`. | `const int gridNy = 32;` |
| `gridNz` | Positive integer | Number of toroidal grid points. | | `const int gridNz = 96;` |
| `ppcNums` | Positive integer | Average number of particles per species in each grid cell. | | `const int ppcNums = 256;` |

## Toroidal Range and Initial Perturbation

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `tubes` | Positive integer | Simulate `1/tubes` of the toroidal domain. | | `const int tubes = 6;` |
| `leftN` | Nonnegative integer | Lower bound of the retained toroidal mode number in the `1/tubes` toroidal domain. | The actual toroidal mode range is `tubes*[leftN,rightN]`, with spacing `tubes`. | `const int leftN = 1;` |
| `rightN` | Nonnegative integer | Upper bound of the retained toroidal mode number in the `1/tubes` toroidal domain. | | `const int rightN = 6;` |
| `refinedTimes` | Positive integer | Field-aligned interpolation refinement factor used by `selectNM_*` poloidal filtering. | The refined grid size is `gridNy * refinedTimes`. | `const int refinedTimes = 32;` |
| `perturbLeftN` | Nonnegative integer | Lower bound of the initial perturbation toroidal mode number in the `1/tubes` toroidal domain. | The actual toroidal mode range is `tubes*[perturbLeftN,perturbRightN]`, with spacing `tubes`. | `const int perturbLeftN = 1;` |
| `perturbRightN` | Nonnegative integer | Upper bound of the initial perturbation toroidal mode number in the `1/tubes` toroidal domain. | | `const int perturbRightN = 6;` |
| `perturbRadialIndex` | `1` to `gridNx` | Radial peak grid point of the initial Gaussian perturbation. | | `const int perturbRadialIndex = 85;` |
| `perturbWidth` | Positive real number | Radial width of the initial Gaussian perturbation. | | `const mhdReal perturbWidth = 0.04;` |
| `perturbAmplitude` | Real number | Normalized `Phi` amplitude of the initial Gaussian perturbation. | | `const mhdReal perturbAmplitude = 2.5e-9;` |

## Filtering

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `ifFilterN_<MHDField>` | `trueType` / `falseType` | Whether to apply toroidal filtering to the MHD field. | | `using ifFilterN_Phi = trueType;` |
| `ifFilterN_dP` | `trueType` / `falseType` | Whether to apply toroidal filtering to the PIC perturbed pressure. | Applies to `dPi/dPa/dPb`. | `using ifFilterN_dP = trueType;` |
| `removeN_<MHDField>` | `{}` or list of toroidal mode numbers | Remove the specified toroidal mode numbers from the MHD field. | | `constexpr std::array<int, 1> removeN_Phi = {7};` |
| `removeN_dP` | `{}` or list of toroidal mode numbers | Remove the specified toroidal mode numbers from the PIC perturbed pressure. | Applies to `dPi/dPa/dPb`. | `constexpr std::array<int, 0> removeN_dP = {};` |
| `selectNM_<MHDField>` | `{{{N, leftM, rightM}, ...}}` | Apply poloidal filtering to the specified toroidal mode numbers of the MHD field. | | `constexpr std::array<std::tuple<int, int, int>, 1> selectNM_Phi = {{{0, 0, 0}}};` |
| `selectNM_dP` | `{{{N, leftM, rightM}, ...}}` | Apply poloidal filtering to the specified toroidal mode numbers of the PIC perturbed pressure. | Applies to `dPi/dPa/dPb`. | `constexpr std::array<std::tuple<int, int, int>, 1> selectNM_dP = {{{0, 0, 1}}};` |

## MHD Physics Switches

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `ifNonlinearMHD` | `trueType` / `falseType` | Whether to include MHD nonlinear terms. | | `using ifNonlinearMHD = trueType;` |
| `ifEparallel` | `trueType` / `falseType` | Whether to include the parallel electric-field term. | | `using ifEparallel = trueType;` |
| `ifFLRMHD` | `trueType` / `falseType` | Whether to include the FLR effect from background thermal ions in the Poisson equation. | | `using ifFLRMHD = falseType;` |
| `ifQNeutrality` | `trueType` / `falseType` | Whether to compute the electron density perturbation from quasi-neutrality. | When enabled, `dNe` is determined by the vorticity term and perturbed densities of enabled species. | `using ifQNeutrality = trueType;` |
| `ifMaxwellStress` | `trueType` / `falseType` | Whether to include Maxwell stress. | Effective only when `ifNonlinearMHD=trueType`. | `using ifMaxwellStress = trueType;` |
| `ifReynoldsStress` | `trueType` / `falseType` | Whether to include Reynolds stress. | Effective only when `ifNonlinearMHD=trueType`. | `using ifReynoldsStress = trueType;` |
| `MaxwellStressCoef` | Real number | Maxwell stress coefficient. | Effective only when `ifMaxwellStress=trueType`. | `const mhdReal MaxwellStressCoef = 1.0;` |
| `ReynoldsStressCoef` | Real number | Reynolds stress coefficient. | Effective only when `ifReynoldsStress=trueType`. | `const mhdReal ReynoldsStressCoef = 1.0;` |

## Dissipation

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `ifNablaPerp2<MHDField>` | `trueType` / `falseType` | Enable second-order perpendicular MHD-field dissipation. | | `using ifNablaPerp2Phi = trueType;` |
| `perp2<MHDField>` | Positive real number | Coefficient for second-order perpendicular MHD-field dissipation. | | `const mhdReal perp2Phi = 1.0e-7;` |
| `ifNablaPara4<MHDField>` | `trueType` / `falseType` | Enable fourth-order parallel MHD-field dissipation. | | `using ifNablaPara4Phi = trueType;` |
| `para4<MHDField>` | Positive real number | Coefficient for fourth-order parallel MHD-field dissipation. | | `const mhdReal para4Phi = 1.0e-6;` |
| `ifNablaPerp2dP` | `trueType` / `falseType` | Enable second-order perpendicular PIC-pressure dissipation. | Applies to `dPi/dPa/dPb`. | `using ifNablaPerp2dP = trueType;` |
| `perp2dP` | Positive real number | Coefficient for second-order perpendicular PIC-pressure dissipation. | Applies to `dPi/dPa/dPb`. | `const mhdReal perp2dP = 1.0e-6;` |
| `ifNablaPara4dP` | `trueType` / `falseType` | Enable fourth-order parallel PIC-pressure dissipation. | Applies to `dPi/dPa/dPb`. | `using ifNablaPara4dP = falseType;` |
| `para4dP` | Positive real number | Coefficient for fourth-order parallel PIC-pressure dissipation. | Applies to `dPi/dPa/dPb`. | `const mhdReal para4dP = 0.0;` |

# PIC Physics Switches

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `ifFLRPIC` | `trueType` / `falseType` | Whether to enable the FLR effect in the PIC part. | | `using ifFLRPIC = trueType;` |
| `ifNonlinearPIC` | `trueType` / `falseType` | Whether to enable PIC nonlinear terms. | | `using ifNonlinearPIC = trueType;` |
| `gyroNums` | Positive integer | Number of gyro-average points. | | `const int gyroNums = 4;` |

# PIC Species Parameters

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `if<Species>` | `trueType` / `falseType` | Whether to enable the corresponding ion species. | | `using ifIon = trueType;` |
| `<Species>Type` | `Maxwell` / `Slowing0` / `Slowing1` / `Slowing2` / `Slowing3` | Velocity distribution type. | | `const disType IonType = Maxwell;` |
| `<Species>Space` | `spaceReal` / `spaceUniform` | Spatial sampling method: marker positions follow the physical spatial distribution or a uniform distribution. | | `const spaceType IonSpace = spaceReal;` |
| `<Species>Velocity` | `velocityReal` / `velocityUniform` | Velocity sampling method: marker velocities follow the physical velocity-space distribution or a uniform distribution. | | `const velocityType IonVelocity = velocityUniform;` |
| `<Species>Mass` | Positive real number | Ion mass, normalized by the proton mass. | | `const picReal IonMass = 2.5;` |
| `<Species>Char` | Positive real number | Ion charge, normalized by the electron charge. | | `const picReal IonChar = 1.0;` |
| `<Species>Beta` | Positive real number | Ion beta on the magnetic axis ( `P/(B0^2/(2*mu0))` ). | | `const picReal IonBeta = 0.037793721898356;` |
| `<Species>Vmin` | Positive real number | Lower bound of the ion velocity, normalized by `VA0` . | | `const picReal IonVmin = 0.0135;` |
| `<Species>Vmax` | Positive real number | Upper bound of the ion velocity, normalized by `VA0` . | | `const picReal IonVmax = 0.54;` |
| `<Species>Vb` | Positive real number | Cutoff velocity of the slowing-down distribution. | | `const picReal BeamVb = 0.1;` |
| `<Species>DeltaV` | Positive real number | Cutoff width of the slowing-down distribution. | | `const picReal BeamDeltaV = 0.1;` |
| `<Species>Lambda0` | Positive real number | Center of `Lambda` in the anisotropic distribution. | | `const picReal BeamLambda0 = 0.4;` |
| `<Species>DeltaLambda2` | Positive real number | Square of the `Lambda` width in the anisotropic distribution. | | `const picReal BeamDeltaLambda2 = 1.0 / (4.5 * 4.5);` |

| Distribution Type | Formula |
|---|---|
| `Maxwell` | `f ~ n * T^(-3/2) * exp[-m*v^2/(2*T)]` |
| `Slowing0` | `f ~ n / (v^3 + v_c^3)` |
| `Slowing1` | `f ~ n * [1 + erf((V_b - v)/DeltaV)] / (v^3 + v_c^3)` |
| `Slowing2` | `f ~ n * exp[-(Lambda - Lambda_0)^2 / DeltaLambda^2] / (v^3 + v_c^3)` |
| `Slowing3` | `f ~ n * [1 + erf((V_b - v)/DeltaV)] * exp[-(Lambda - Lambda_0)^2 / DeltaLambda^2] / (v^3 + v_c^3)` |

# Diagnostic Switches

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `ifDiagAmplitude` | `trueType` / `falseType` | Whether to diagnose mode amplitude. | | `using ifDiagAmplitude = trueType;` |
| `ifDiagFrequency` | `trueType` / `falseType` | Whether to diagnose mode frequency. | | `using ifDiagFrequency = trueType;` |
| `ifDiagEparallel` | `trueType` / `falseType` | Whether to diagnose the parallel electric field. | | `using ifDiagEparallel = trueType;` |
| `ifDiagDensity` | `trueType` / `falseType` | Whether to diagnose ion perturbed density. | | `using ifDiagDensity = trueType;` |
| `ifDiagDiffusivity` | `trueType` / `falseType` | Whether to diagnose ion diffusivity. | | `using ifDiagDiffusivity = trueType;` |
| `ifDiagZFDrive` | `trueType` / `falseType` | Whether to diagnose the zonal-flow drive source. | | `using ifDiagZFDrive = falseType;` |
| `ifCheckNAN` | `trueType` / `falseType` | Whether to check for NaN. | If NaN is diagnosed, the program stops immediately and writes output. | `using ifCheckNAN = trueType;` |

# MHD Field Output Switches

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `ifOutputPhi` | `trueType` / `falseType` | Whether to output `Phi` at multiple time points. | | `using ifOutputPhi = trueType;` |
| `ifOutputA` | `trueType` / `falseType` | Whether to output `A` at multiple time points. | | `using ifOutputA = falseType;` |
| `ifOutputdNe` | `trueType` / `falseType` | Whether to output `dNe` at multiple time points. | | `using ifOutputdNe = falseType;` |
| `ifOutputdTe` | `trueType` / `falseType` | Whether to output `dTe` at multiple time points. | | `using ifOutputdTe = falseType;` |
| `ifOutputdPi` | `trueType` / `falseType` | Whether to output `dPi` at multiple time points. | | `using ifOutputdPi = falseType;` |
| `ifOutputdPa` | `trueType` / `falseType` | Whether to output `dPa` at multiple time points. | | `using ifOutputdPa = falseType;` |
| `ifOutputdPb` | `trueType` / `falseType` | Whether to output `dPb` at multiple time points. | | `using ifOutputdPb = falseType;` |

# Phase-space Output Parameters

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `gridE` | Positive integer | Number of phase-space grid points in the `E` direction. | | `const int gridE = 96;` |
| `gridPphi` | Positive integer | Number of phase-space grid points in the `Pphi` direction. | | `const int gridPphi = 128;` |
| `gridLambda` | Positive integer | Number of phase-space grid points in the `Lambda` direction. | | `const int gridLambda = 48;` |
| `ppcPhase` | Positive integer | Average number of particles per species in each phase-space grid cell. | | `const int ppcPhase = 2048;` |
| `ifOutputPhaseSpaceJacobian` | `trueType` / `falseType` | Whether to output the phase-space Jacobian. | Output during the initialization stage. | `using ifOutputPhaseSpaceJacobian = trueType;` |
| `ifOutputPhaseSpaceOrbit` | `trueType` / `falseType` | Whether to output phase-space orbit frequencies. | Requires the `<Species>PhaseSpaceMapping` file. | `using ifOutputPhaseSpaceOrbit = falseType;` |
| `ifOutputPhaseSpaceF0` | `trueType` / `falseType` | Whether to output the phase-space equilibrium distribution function. | Output during the initialization stage. | `using ifOutputPhaseSpaceF0 = falseType;` |
| `ifOutputPhaseSpaceDeltaF` | `trueType` / `falseType` | Whether to output the phase-space perturbed distribution function. | | `using ifOutputPhaseSpaceDeltaF = falseType;` |
| `ifOutputPhaseSpacePower` | `trueType` / `falseType` | Whether to output phase-space wave-particle interaction power. | | `using ifOutputPhaseSpacePower = falseType;` |
| `gridVpara` | Positive integer | Number of pitch-space grid points in the `vpara` direction. | | `const int gridVpara = 128;` |
| `gridVperp` | Positive integer | Number of pitch-space grid points in the `vperp` direction. | | `const int gridVperp = 64;` |
| `ppcPitch` | Positive integer | Average number of particles per species in each pitch-space grid cell. | | `const int ppcPitch = 2048;` |
| `ifOutputPitchSpaceJacobian` | `trueType` / `falseType` | Whether to output the pitch-space Jacobian. | Output during the initialization stage. | `using ifOutputPitchSpaceJacobian = trueType;` |
| `ifOutputPitchSpaceF0` | `trueType` / `falseType` | Whether to output the pitch-space equilibrium distribution function. | Output during the initialization stage. | `using ifOutputPitchSpaceF0 = falseType;` |
| `ifOutputPitchSpaceDeltaF` | `trueType` / `falseType` | Whether to output the pitch-space perturbed distribution function. | | `using ifOutputPitchSpaceDeltaF = falseType;` |
| `ifOutputPitchSpacePower` | `trueType` / `falseType` | Whether to output pitch-space wave-particle interaction power. | | `using ifOutputPitchSpacePower = falseType;` |

# Time Advancement

| Parameter | Allowed Values | Description | Notes | Example |
|---|---|---|---|---|
| `ifContinue` | `trueType` / `falseType` | Whether to start from continue files. | The `inputDir` directory must contain the `MHDContinue` and `PICContinue` continue files. They come from the previous run's `outputDir/final`. | `using ifContinue = falseType;` |
| `continueSteps` | Nonnegative integer | Number of steps already completed before resuming. | Must match the suffix of the continue file names. | `const int continueSteps = 0;` |
| `dt` | Positive real number | MHD time step. | | `const double dt = 0.02;` |
| `totalSteps` | Positive integer | Total number of MHD steps in this run. | | `const int totalSteps = 20000;` |
| `ratioDt` | Positive integer | Ratio of the PIC time step to the MHD time step. | PIC uses `dt * ratioDt` for each push. | `const int ratioDt = 1;` |
| `sortSteps` | Positive integer | Particle sorting interval. | | `const int sortSteps = 25;` |
| `diagSteps` | Positive integer | Diagnostic sampling interval. | | `const int diagSteps = 1;` |
| `outputSteps` | Positive integer | Field and phase-space output interval. | | `const int outputSteps = 2500;` |